
TP2 BDA: Requêtes, dépendances fonctionnelles et normalisation

ING2 INFORMATIQUE
Sup Galilée

Réalisé par
Daniel THARMARAJAH 12000230

Contents

Exercice n°1	3
1.	3
2.	3
3.	3
4.	3
5.	3
6.	3
7.	3
8.	4
9.	4
10.	4
11.	4
12.	4
13.	4
14.	4
15.	5
16.	5
17.	5
18.	5
Exercice n°2	5
1.	5
2.	5
3.	6
Exercice n°3	6
1.	6
2.	6
a.	6
b.	6
c.	6
3.	6
Exercice n°4	6
1.	6
2.	6
3.	7
4.	7
5.	7
6.	7
7.	7
8.	8
9.	8
10.	8
11.	8
12.	8
13.	9
14.	9

Exercice n°1

1.

```
SELECT * from department WHERE budget IN (SELECT max(budget) FROM department) ;
```

2.

```
SELECT name,salary from teacher WHERE salary >= (SELECT avg(salary) FROM teacher) ;
```

3.

```
SELECT teacher_name,teacher_id,student_id,student_name,  
count(course_id) as nb_courses  
FROM  
(SELECT id as student_id,name as student_name,course_id,sec_id,semester,year FROM student  
NATURAL JOIN takes NATURAL JOIN section)  
NATURAL JOIN  
(SELECT name as teacher_name,id as teacher_id,course_id,sec_id,semester,year  
FROM teaches NATURAL JOIN teacher)  
GROUP BY teacher_name,teacher_id,student_id,student_name  
HAVING count(course_id) >= 2
```

4.

```
SELECT * FROM (SELECT teacher_name,teacher_id,student_id,student_name,count(course_id) as nb_courses  
FROM  
(SELECT id as student_id,name as student_name,course_id,sec_id,semester,year  
FROM student NATURAL JOIN takes NATURAL JOIN section) NATURAL JOIN  
(SELECT name as teacher_name,id as teacher_id,course_id,sec_id,semester,year  
FROM teaches NATURAL JOIN teacher )  
GROUP BY teacher_name,teacher_id,student_id,student_name )  
WHERE nb_courses >= 2
```

5.

```
SELECT * FROM (SELECT name,id FROM takes NATURAL JOIN student NATURAL JOIN section ) MINUS  
(SELECT name,id FROM takes NATURAL JOIN student NATURAL JOIN section WHERE year = 2010)
```

6.

```
SELECT * FROM teacher WHERE name LIKE 'E%'
```

7.

```
SELECT * FROM teacher WHERE salary = (SELECT min(salary)  
FROM teacher A WHERE (SELECT count(*) FROM teacher B WHERE A.salary < B.salary) < 4 ) ;
```

8.

```
SELECT * FROM teacher A WHERE (SELECT count(*) as nb FROM teacher B WHERE A.salary > B.salary)
<= 2
ORDER BY salary DESC;
```

9.

```
SELECT * FROM student WHERE id IN (SELECT id FROM student NATURAL JOIN takes
WHERE semester = 'Fall' AND year = 2009 )
```

10.

```
SELECT * FROM student WHERE id = SOME (SELECT id FROM student NATURAL JOIN takes
WHERE semester = 'Fall' AND year = 2009 )
```

11.

```
SELECT DISTINCT(name)
FROM student NATURAL INNER JOIN takes
WHERE semester = 'Fall' AND year = 2009
```

12.

```
SELECT DISTINCT(name)
FROM student NATURAL INNER JOIN takes
WHERE
EXISTS
(SELECT name FROM student NATURAL INNER JOIN takes WHERE semester = 'Fall' AND year = 2009 )
```

13.

```
SELECT *
FROM (SELECT id,name,course_id,sec_id,semester,year FROM student natural join takes) A
INNER JOIN
(SELECT id,name,course_id,sec_id,semester,year FROM student natural join takes) B
ON
A.course_id = B.course_id and A.semester = B.semester
and A.sec_id = B.sec_id and A.year = B.year
WHERE
A.id <> B.id
```

14.

```
SELECT teacher.name, count(*)
FROM (takes
      INNER JOIN teaches USING(course_id, sec_id, semester, year))
      INNER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY count(*) DESC;
```

15.

```
SELECT teacher.name, count(course_id)
FROM (takes
      INNER JOIN teaches USING(course_id, sec_id, semester, year))
      RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY count(course_id) DESC;
```

16.

```
WITH mytakes (id, course_id, sec_id, semester, year, grade) AS (
  SELECT id, course_id, sec_id, semester, year, grade
  FROM takes
  WHERE grade = 'A'
)
SELECT teacher.name, count(course_id)
FROM (mytakes
      INNER JOIN teaches USING(course_id, sec_id, semester, year))
      RIGHT OUTER JOIN teacher ON teaches.id = teacher.id
GROUP BY teacher.name, teacher.id
ORDER BY count(course_id) DESC;
```

17.

```
SELECT teacher.name, student.name, count(*)
FROM (teacher NATURAL JOIN teaches)
INNER JOIN (takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
GROUP BY teacher.name, student.name;
```

18.

```
SELECT mytable.tn, mytable.sn
FROM (
  SELECT teacher.name AS tn, student.name AS sn, count(*) AS tc
  FROM (teacher NATURAL JOIN teaches)
  INNER JOIN (takes NATURAL JOIN student) USING (course_id, sec_id, semester, year)
  GROUP BY teacher.name, student.name
) AS mytable
WHERE tc >= 2;
```

Exercice n°2

1.

R n'est pas en 3NF donc on va decomposer en 2 relations R1(A,B) et R2(B,C) et chacune de ces relations est en 3FN

2.

Elle est déjà en 3NF

3.

La dépendance $C \rightarrow B$ viole BCNF donc on décompose R en 2 relations : $R_1(A,C)$ et $R_2(C,B)$ et cette décomposition est en BCNF

Exercice n°3

1.

$$F = \left\{ \begin{array}{l} (\{C\} \rightarrow \{C\}), \\ (\{B\} \rightarrow \{D\}), (\{B\} \rightarrow \{B\}), (\{B\} \rightarrow \{D, B\}), \\ (\{A\} \rightarrow \{C\}), (\{A\} \rightarrow \{B\}), (\{A\} \rightarrow \{A\}), (\{A\} \rightarrow \{D\}), (\{A\} \rightarrow \{E\}), \\ (\{A\} \rightarrow \{C, B\}), (\{A\} \rightarrow \{A, C\}), (\{A\} \rightarrow \{D, C\}), (\{A\} \rightarrow \{E, C\}), \\ (\{A\} \rightarrow \{A, B\}), (\{A\} \rightarrow \{D, B\}), (\{A\} \rightarrow \{E, B\}) \end{array} \right\}$$

2.

a.

$$\{A, B\}^+ = \{A, B, C, D, E\}$$

b.

$$\{A, F\}^+ = \{A, B, C, D, E, F\} = R$$

c.

La relation R n'est pas en BCNF. Nous la décomposons donc de la manière suivante :

$$R_1(A, B, C, D, E) \quad \text{et} \quad R_2(A, F)$$

3.

Donc, $\{A, F\}$ est une **superclé** de R .

Exercice n°4

1.

```
def printDependencies(F):
    for alpha, beta in F:
        print("\t", alpha, "---->", beta)
```

2.

```
def printRelations(T):
    for R in T:
        print("\t", R)
```

3.

```
def powerSet(inputset):
    _result = []
    for r in range(1, len(inputset) + 1):
        _result += map(set, itertools.combinations(inputset, r))
    return _result
```

4.

```
def fermetureAttributs(F, K):
    K_plus = set(K)
    size = 0
    while size != len(K_plus):
        size = len(K_plus)
        for alpha, beta in F:
            if alpha.issubset(K_plus):
                K_plus.update(beta)
    return K_plus
```

5.

```
def clotureDependances(F):
    R = set()
    for alpha, beta in F:
        R.update(alpha | beta)
    F_plus = []
    for K in powerSet(R):
        for beta in powerSet(fermetureAttributs(F, K)):
            F_plus.append([K, beta])
    return F_plus
```

6.

```
def isDependency(F, alpha, beta):
    return beta.issubset(fermetureAttributs(F, alpha))
```

7.

```
def isSuperKey(F, R, K):
    return R.issubset(fermetureAttributs(F, K))
```

8.

```
def isCadidateKey(F, R, K):
    if not isSuperKey(F, R, K):
        return False
    for K_prime in K:
        K_prime2 = set(K).discard(K_prime)
        if isSuperKey(F, R, K_prime2):
            return False
    return True
```

9.

```
def listCandidateKeys(F, R):
    candKeys = []
    for K in powerSet(R):
        if isCadidateKey(F, R, K):
            candKeys.append(K)
    return candKeys
```

10.

```
def listSuperKeys(F, R):
    superKeys = []
    for K in powerSet(R):
        if isSuperKey(F, R, K):
            superKeys.append(K)
    return superKeys
```

11.

```
def oneCandidateKey(F, R):
    K = set(R)
    while not isCadidateKey(F, R, K):
        for A in K:
            if isCadidateKey(F, R, K - {A}):
                K.discard(A)
                break
```

12.

```
def isBCNFRelation(F, R):
    for K in powerSet(R):
        K_plus = fermetureAttributs(F, K)
        Y = K_plus.difference(K)
        if not R.issubset(K_plus) and not Y.isdisjoint(R):
            return False, [K, Y & R]
    return True, [{}, {}]
```


13.

```
def isBCNFRelations(F, T):  
    for R in T:  
        if isBCNFRelation(F, R) == False:  
            return False
```

14.

```
def computeBCNFDecomposition(F, T):  
    OUT, size = list(T), 0  
    while size != len(OUT):  
        size = len(OUT)  
        for R in OUT:  
            _isR_BCNF, [alpha, beta] = isBCNFRelation(F, R)  
            if _isR_BCNF == False:  
                if alpha | beta not in OUT:  
                    OUT.append(alpha | beta)  
                if R.difference(beta) not in OUT:  
                    OUT.append(R.difference(beta))  
                OUT.remove(R)  
                break  
    return OUT
```